

חישוביות - שיעור 12

דוד קלוטה

21 ביוני 2026

הקדמה

נדבר על המחלקה CoNL.

התחלנו!

12.1 המחלקות CoNL, CoNP

הגדרה 12.1.1 (המחלקה CoNL). נגיד ששפה A שייכת למחלקה CoNL אם $\bar{A} \in \text{NL}$.

הגדרה 12.1.2 (המחלקה CoNP). נגיד ששפה A שייכת למחלקה CoNP אם $\bar{A} \in \text{NP}$.

טענה 12.1.1. 1. אם $A \leq_P B$ וגם $B \in \text{CoNP}$ אז גם $A \in \text{CoNP}$.

2. אם $A \leq_L B$ וגם $B \in \text{CoNL}$ אז גם $A \in \text{CoNL}$.

הוכחה. יש רדוקציה $A \leq_P B$ אם A אמ"ם יש רדוקציה $\bar{A} \leq_P \bar{B}$ (אותה הרדוקציה) ואז סעיף 1 נובע ממשפט הרדוקציה עבור NP. □
יש רדוקציה $A \leq_L B$ אם A אמ"ם יש רדוקציה $\bar{A} \leq_L \bar{B}$ (אותה הרדוקציה) ואז סעיף 2 נובע ממשפט הרדוקציה עבור NL.

תזכורת (משפט הרדוקציה עבור NL). אם $A \leq_L B$ וגם $B \in \text{NL}$ אז $A \in \text{NL}$.

הגדרה 12.1.3 (CoNP-קושי ושלמות). נגיד ששפה B היא CoNP-קשה אם לכל $A \in \text{CoNP}$ מתקיים $A \leq_P B$.
נגיד ש- B היא CoNP-שלמה אם היא CoNP-קשה וגם $B \in \text{CoNP}$.

הגדרה 12.1.4 (CoNL-קושי ושלמות). נגיד ששפה B היא CoNL-קשה אם לכל $A \in \text{CoNL}$ מתקיים $A \leq_L B$.
נגיד ש- B היא CoNL-שלמה אם היא CoNL-קשה וגם $B \in \text{CoNL}$.

מסקנה 12.1.1. A היא CoNP-שלמה אם $\bar{A}$ היא NP-שלמה. באופן דומה, A היא CoNL-שלמה אם $\bar{A}$ היא NL-שלמה.
דוגמה 12.1.1. SAT היא CoNP-שלמה, ו-PATH היא CoNL-שלמה.

טענה 12.1.2. אם קיימת שפה A שהיא CoNP-שלמה וגם $A \in \text{NP}$ אז $\text{NP} = \text{CoNP}$.

זוהי שאלה פתוחה ומובטח פרס כספי רציני ממדיי.

טענה 12.1.3. אם קיימת שפה A שהיא CoNL-שלמה וגם $A \in \text{NL}$ אז $\text{NL} = \text{CoNL}$.

הוכחה. נשים לב כי $\overline{\text{Path}}$ היא CoNL-שלמה. בפרט, לכל $A \in \text{CoNL}$, מתקיים $A \leq_L \overline{\text{Path}}$. משום שלפי משפט אימרמן-שלפשוני מתקיים כי $\overline{\text{Path}} \in \text{NL}$, לפי משפט הרדוקציה ל-NL מתקיים $A \in \text{NL}$, כלומר $\text{CoNL} \subseteq \text{NL}$.
אם $\text{CoNL} \subseteq \text{NL}$ אז עבור $B \in \text{NL}$ מתקיים כי $\bar{B} \in \text{CoNL}$ ולכן $\bar{B} \in \text{NL}$ כלומר $B \in \text{CoNL}$ ולכן גם $\text{CoNL} \supseteq \text{NL}$. □
סיימנו.

משפט 12.1.1 (משפט אימרמן-שלפשוני Immerman-Szelepcsényi theorem). מתקיים כי $\overline{\text{Path}} \in \text{NL}$.
כלומר, מתקיים כי $\text{Path} \in \text{CoNL}$.

הוכחה. האתגר שלנו יהיה להשתכנע שאין מסלול בין צמתים מסוימים. מראה מכונת M שלכל קלט (G, s, t) יכולה לקבל, לשלול או להיכשל.

מתקיים: אם $(G, s, t) \in \overline{\text{Path}}$ אז $M(\langle G, s, t \rangle) \in \{q_{\text{acc}}, q_{\text{fail}}\}$ ויש לפחות ריצה אחת שמקבלת. אם $(G, s, t) \notin \overline{\text{Path}}$ אז $M(\langle G, s, t \rangle) \in \{q_{\text{rej}}, q_{\text{fail}}\}$ ויש לפחות ריצה אחת שדוחה.
לצורך בניית M נגדיר מכונה M' אי דטרמיניסטית שמספקת במקום $\mathcal{O}(\log n)$. בעבור מכונה זו נידרש למכונת העזר הבאה:

Algorithm 1 Course of action for M''

```
1: procedure CALCULATE( $G, s, c_{i-1}$ )
Require:  $G$  is a directed graph
Require:  $s \in V(G)$  is a vertex
Require:  $c_{i-1} \in \mathbb{N}$ 
2:   Let  $c_i \leftarrow 0$ 
3:   for each  $v \in V(G)$  do
4:     Let  $d \leftarrow 0$ 
5:     for each  $u \in V(G)$  do
6:       Guess a path from  $s$  to  $u$  of length  $\leq i - 1$ 
7:       if guessed successfully then
8:          $d \leftarrow d + 1$ 
9:         if  $(u, v) \in E(G)$  then
10:           $c_i \leftarrow c_{i-1}$ 
11:          Exit For
12:        end if
13:      end if
14:    end for
15:    if the For loop completed and  $d \neq c_{i-1}$  then
16:      Fail
17:    end if
18:  end for
19:  return  $c_i$ 
20: end procedure
```

נוכיח נכונות: לכל צומת v מועמד להיספר ב- c_i כצומת במרחק $i \leq m$ נעבור על כל המועמדים u להיות הצומת במקום הלפני אחרון במסלול. לכל מועמד כזה, אם מצליחים להגיע אליו, מעלים את המונה d ב-1.

מסקנה 12.1.2. בריצה שבה תמיד ננחש מסלול באורך $i - 1 \leq$ לכל צומת u שיש לו מסלול כזה, נקבל $d = c_{i-1}$ אם אפשר להגיע ל- v , נגיע אליו. כלומר בתום ריצה M'' הערך c_i יהיה נכון.

מצד שני, אם בתהליך ספירת c_i (מספר צמתים v במרחק $i \leq$) יש צומת שלא ספרנו והיה צריך להיספר, זה אומר שיש שכן שלו u שהיינו צריכים להגיע אליו ולא הגענו, כלומר בתום לולאה הפנימית הרלוונטית $d \neq c_{i-1}$ והמכונה תיכשל.

מבחינת מקום ריצה, נצטרך מונים D, c_i, c_{i-1} ומקום לניחוש המסלול מ- s ל- u ומונה על אורך המסלול הנ"ל ומוודא ש- $i - 1 \leq$. כל אחד מהנ"ל לוקח מקום $\mathcal{O}(\log n)$, סה"כ $\mathcal{O}(\log n)$.

בבנה כעת את M' באופן הבא:

Algorithm 2 Course of action for M'

```
1: procedure ALL-REACHABLE( $G, s$ )
Require:  $G$  is a directed graph
Require:  $s \in V(G)$ 
Ensure: The tape contains the total nodes reachable from  $s$  in  $G$ 
2:   Let  $(c_i \leftarrow 1)_{i \in [n] \cup \{0\}}$  be a vector of integers
3:   Let  $j \leftarrow 1$ 
4:   while  $j \leq n - 1$  do
5:      $c_i \leftarrow \text{CALCULATE}(G, s, c_{i-1})$ 
6:      $i \leftarrow i + 1$ 
7:   end while
8:   return CALCULATE( $G, s, c_{n-1}$ ) if successful, fail otherwise
9: end procedure
```

נשים לב שבמכונה M , אם נשנה את q_{fail} ל- q_{rej} , נקבל בדיוק מכונת NL עבור $\overline{\text{PATH}}$. הערה: ההיפוך $q_{\text{rej}} \leftrightarrow q_{\text{acc}}$ במכונת NL של PATH נותן מכונה שכאשר $(G, s, t) \notin \text{PATH}$ עלולה לפעמים להחזיר True.

הערה 12.1.1. מימוש נאיבי של אלגוריתם BFS / DFS או כל אלגוריתם שעובר על כל המסלולים עלול לדרוש מקום $\Theta(|E(G)|)$ ובפרט לא מקום $\mathcal{O}(\log n)$.

בהינתן מכונה M' נוכל לבנות את המכונה M באופן הבא:

Algorithm 3 Course of action for M

```

1: procedure NOT-PATH( $G, s, t$ )
Require:  $G$  is a directed graph
Require:  $s, t \in V(G)$ 
Ensure: there is no directed path from  $s$  to  $t$  in  $G$ 
2:   Let  $a \leftarrow \text{ALL-REACHABLE}(G, s)$ 
3:   Let  $b \leftarrow \text{ALL-REACHABLE}((V(G) \setminus \{t\}, E(G) \setminus \{(x, y) \in E(G) \mid x = t \vee y = t\}), s)$ 
4:   return  $a = b$ 
5: end procedure

```

אם $(G, s, t) \in \overline{\text{PATH}}$, אז אין מסלול מ- s ל- t ולכן התוצאות משני הגרפים הן אותו המספר. אחרת, $(G, s, t) \notin \overline{\text{PATH}}$ כלומר $(G, s, t) \in \text{PATH}$ אז יש מסלול מ- s ל- t ולכן שני המספרים ייצאו שונים, והמכונה תדחה או תיכשל (כי M' או מחזירה את המספר הנכון או נכשלת).

מבחינת זמן ריצה, חישוב המספרים הוא $\mathcal{O}(\log n)$ והשוואתן היא גם $\mathcal{O}(\log n)$. $\square$

דוגמה 12.1.2 (דוגמה לשימוש במשפט אימרמן-שלפשי). נגדיר את השפה הבאה:

$$\text{STRONGLY-CONNECTED} \stackrel{\text{def}}{=} \left\{ \langle G \rangle \mid \begin{array}{l} G \text{ is a directed graph, and for each} \\ \text{two vertices } u, v \in V(G) \text{ there exists} \\ \text{a directed path from } u \text{ to } v \end{array} \right\}$$

טענה 12.1.4. $\text{STRONGLY-CONNECTED} \in \text{NL}$.

הוכחה. נראה מכונת טיורינג לא דטרמיניסטית שמסתפקת במקום לוגריתמי:

Algorithm 4 Nondeterministic TM for STRONGLY – CONNECTED

```

1: procedure IS-STRONGLY-CONNECTED( $G$ )
Require:  $G$  is a directed graph
2:   if  $|V(G)| \leq 1$  then
3:     return True
4:   end if
5:   for each pair  $(u, v) \in (V(G))^2$  where  $u \neq v$  do
6:     Nondeterministically guess a path from  $u$  to  $v$ 
7:     if there is no such path then
8:       return False
9:     end if
10:  end for
11:  return True
12: end procedure

```

נוכיח נכונות: אם $G \in \text{STRONGLY-CONNECTED}$ אז לכל (u, v) שונות יש מסלול מכוון מ- u ל- v ולכן אם הפונקציה IS-STRONGLY-CONNECTED תנחש כל פעם מסלול כזה, אז הריצה תחזיר True. אם $G \notin \text{STRONGLY-CONNECTED}$, אז קיימים $u, v \in V(G)$ כך שאין מסלול מכוון מ- u ל- v , ולכן הריצה על G תחזיר False. מבחינת סיבוכיות מקום, M צריכה מקום עבור u , עבור v (מונים שרצים על כל זוגות הצמתים). וכן, מקום עבור צומת נוכחי וצומת הבא בטיול (תזכורת: אין צורך לשמור את הלטיל, רק מקום נוכחי ולכן עוברים). סה"כ מקום $\mathcal{O}(\log n)$. $\square$

נגדיר את השפה הבאה:

$$\text{AL2SCC} = \text{AT-LEAST-2-STRONGLY-CONNECTED-COMPONENTS}$$

$$\stackrel{\text{def}}{=} \{ \langle G \rangle \mid G \text{ contains at least two strongly connected components} \}$$

טענה 12.1.5. $\text{AL2SCC} \in \text{NL}$.

הוכחה. המשלימה של השפה הנ"ל (בניכוי קלטים לא בפורמט) היא השפה STRONGLY – CONNECTED ונשתמש בטענה 12.1.3 ובמשפט אימרמן-שלפשי. $\square$

12.2 חיפוש פתרון אופטימלי

ראינו שהשפה

$$\text{E3SAT} = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable CNF formula with exactly 3 literals in each clause}\}$$

היא NP-שלמה.

נעיין בשתי השאלות הבאות:

1. איך נעבור מהכרעה לחיפוש?

2. האם אפשר למצוא פתרון מקורב?

טענה 12.2.1. אם יש אלגוריתם פולינומי דטרמיניסטי שמכריע את SAT, אז יש גם אלגוריתם פולינומי דטרמיניסטי שבהינתן נוסחה ϕ מוצא עבורה הצבה מספקת הנקרא "רדוקציה עצמית" - רדוקציה מבעיית חיפוש לבעיית הכרעה.

הוכחה. נתונה מכונת טיורינג M דטרמיניסטית פולינומית שמכריעה את SAT. בהינתן נוסחה ϕ מעל המשתנים $(x_i)_{i \in [n]}$ נרצה למצוא הצבה מספקת, משתנה אחרי משתנה. M' תהיה מכונת טיורינג דטרמיניסטית פולינומית שיוצרת את ההצבה המספקת באופן הבא:

Algorithm 5 Course of action of M'

```

1: procedure BUILD-SATIFYING-ASSIGNMENT( $\phi$ )
Require:  $\phi$  is a satisfiable CNF formula
Ensure: the result of this function is an assignment which satisfies  $\phi$ 
2:   Let  $i \leftarrow 1$ 
3:   while  $i \leq n$  do
4:     Assign True to  $x_i$ 
5:     if  $\phi$  is not satisfiable with the current assignments then
6:       Assign False to  $x_i$ 
7:     end if
8:   return the assignment
9: end while
10: end procedure
```

איך נאלץ את $x_i = \text{True}$? כל פסוקית עם הליטרל x_i - נמחק אותה (כי היא כבר מסופקת) וכל פסוקית עם הליטרל $\bar{x}_i$ נמחק את הליטרל מהפסוקית (כי הוא לא יספק אותה) באופן דומה אפשר לקבוע $x_i = \text{False}$ כלומר $\bar{x}_i = \text{True}$. נראה נכונות זמן ריצה:

אם ϕ ספיקה, אז בכל שלב M' קסבעה ערך למשתנה נוסף שמתאים להצבה מספקת, כלומר הנוסחה החדשה עדיין ספיקה. בתום התהליך, קבענו ערך לכל המשתנים ונשארו עם נוסחה ריקה (ללא פסוקיות). לכן ההצבה שהתקבלה מספקת את הנוסחה המקורית ϕ . זמן הריצה של האלגוריתם: מספר השלבים כמספר המשתנים. כל שלב מפעילים את M שהיא מכונה דטרמיניסטית פולינומית על קלט באורך קטן או שווה לאורך של $|\phi|$ - לכן סה"כ זמן הריצה פולינומי. $\square$

נרצה לעשות רדוקציה עצמית עבור E3SAT:

הוכחה. הרעיון דומה. כלומר, נריץ את M' כמו בבנייה הקודמת. הבעיה שהנוסחה המתקבלת בכל שלב עלולה להיות שלא מתצורת E3CNF. מכונת ההכרעה M יודעת להכריע את E3SAT - כלומר לקבוע ספיקות, אבל רק כשהקלט מתצורת E3CNF. ידוע ש-E3SAT היא NP-שלמה, כלומר קיימת רדוקציה פולינומית $\text{SAT} \leq_P \text{E3SAT}$. נקרא למכונת הרדוקציה הנ"ל M_f . נריץ את M' כמו בהוכחת טענה 12.2.1, אבל לכל נוסחה זמנית שנקבל על ידי קיבוע של x_i נוסף, לפני שנעביר אותה למכונת ההכרעה M נפעיל עליה את M_f .

הוכחת נכונות - כמו קודם (M_f משמרת את תכונת הספיקות). זמן הריצה: יש מספר שלבים כמספר המשתנים, בכל שלב הנוסחה המתקבלת (לפני הפעלת M_f) היא באורך $|\phi| \leq$. הפעלת M_f לוקחת לכל היותר בפולינום, ולכן סה"כ זמן הריצה פולינומי בכל שלב, ומכאן זמן הריצה פולינומי לריצת M' על ϕ . $\square$